

CSCE 650 VIRTUAL REALITY · MODULE 10 · LECTURE NOTES

Rendering for VR

Twice the pixels, half the time.

MODULE OUTCOMES

Apply stereo rendering optimisations and say what each saves.

Explain foveated rendering, fixed and eye-tracked.

Explain variable rate shading and where to apply it.

Choose antialiasing appropriately for VR.

Build a VR performance budget and defend it.

1 Rendering two eyes

Technique	Mechanism	What it saves
Sequential stereo	Render the scene twice, independently, once per eye.	Nothing. The naive baseline, and roughly twice the cost of a single view.
Instanced stereo	Issue one draw call with an instance count of two; a vertex shader selects the eye's view matrix by instance index.	CPU draw call submission — commonly around 40% of CPU render time, which matters because VR is frequently CPU-bound.
Multiview	A hardware and API feature: one submission, and the GPU produces both views, transforming each vertex once and projecting it twice.	CPU submission and a large share of vertex processing. The best of the stereo techniques where it is supported.
Single-pass / double-wide	Both eyes rendered into one wide render target with a viewport per eye.	Render target switches and associated state changes.

WHAT CAN AND CANNOT BE SHARED

Fully shareable between the eyes: shadow map generation, lightmap and probe lookups, most culling, animation and skinning, particle simulation, physics, and every other CPU-side computation. None of this depends on which eye is being rendered.

Partly shareable: vertex processing. Multiview transforms each vertex to world space once and then projects it twice, which recovers most of the duplicated work.

Not shareable at all: fragment shading. The two eyes genuinely see different pixels of different surfaces at different angles, and **that difference is precisely what stereo is**. There is no legitimate way to share it.

So the achievable saving is bounded by how fragment-bound your scene is. A vertex-heavy or draw-call-heavy scene benefits enormously from multiview; a fragment-bound one barely at all. **And most VR scenes are fragment-bound**, because the render targets are large and oversized for distortion (Module 04) — which is why §2's techniques matter more than §1's.

2 Foveated rendering

	Fixed foveated	Eye-tracked foveated
Assumption	The user looks mostly toward the centre of the display.	None — the gaze is measured.
Method	Reduce shading rate in concentric regions toward the edges.	Full rate only in a small region around the measured gaze point.
Hardware	None beyond VRS support.	Eye tracking, with low latency.
Aggressiveness	Conservative — the fovea might be anywhere within the central region, so the full-rate area must be generous.	Aggressive — the fovea's location is known to within a degree.
Typical saving	20–40%	50–70%

FOVEATED RENDERING MUST BEAT THE SACCAD

A saccade — a rapid eye movement between fixation points — takes 20 to 50 ms and can traverse 20 degrees or more. They happen several times a second, continuously, without conscious awareness.

If the high-resolution region updates after the eye has arrived, the user sees low-resolution content at the centre of vision — exactly where acuity is highest and the deficiency is most visible.

And that is far more noticeable than uniformly lower resolution would have been. A brief blur at the fovea is conspicuous in a way that a permanently softer image is not, which means a foveation system that is too slow is worse than no foveation at all.

So the eye tracker's latency plus the rendering pipeline's latency must fit within the saccade, which is a demanding constraint on both. **Saccadic suppression helps:** visual sensitivity is substantially reduced during the movement itself, which buys back some of the budget, and systems exploit it deliberately.

- **Variable rate shading** computes one shaded value per 2×2 , 4×4 , or larger block of pixels, instead of per pixel, in regions where full detail is not required.
- **Hardware support makes it nearly free.** It is a rasteriser feature rather than a post-process, so the saved work is genuinely never done rather than being done and discarded.
- **Drive it by eccentricity** (which is foveation), **by motion** (fast-moving content is perceptually blurred anyway), or **by material** (a flat painted wall needs less shading rate than a detailed normal-mapped surface).
- **Geometry and depth remain at full rate.** Only shading is coarsened, so silhouettes and edges stay sharp — which is why the artefacts are subtle and why VRS degrades gracefully where a straightforward resolution reduction does not.
- **It composes with everything else**, which makes it unusually easy to adopt incrementally: it can be enabled on part of a scene without restructuring the renderer.

3 Where VR reverses the usual advice

Standard practice in flat rendering	In VR
TAA is the default antialiasing.	MSAA frequently wins. TAA accumulates across frames and smears under head motion, which is continuous in VR, and the smear reads as latency (Module 02).
Deferred rendering, for many dynamic lights.	Forward+ or clustered forward frequently wins. See below.
Motion blur adds cinematic realism.	Never use it. The display's low persistence exists to remove motion blur; adding it back increases perceived latency (Module 02).
Post-processing is comparatively cheap.	Doubled by stereo, and VR is frequently bandwidth-bound because of the oversized render targets. Full-screen passes cost more here than the instruction counts suggest.
Dynamic resolution is a nice-to-have.	Essential — see below.

WHY FORWARD RENDERING RETURNED FOR VR

MSAA works with forward rendering and does not work with deferred — and VR wants MSAA for the persistence reasons in Module 02. This alone is close to decisive.

The G-buffer's bandwidth cost is doubled by stereo. A deferred renderer writes several full-screen targets per eye, at render resolutions already inflated 30 to 40% for distortion — and VR is frequently bandwidth-bound rather than arithmetic-bound, particularly on standalone hardware with mobile memory systems.

Transparency is awkward in deferred rendering and requires a separate forward path anyway, and VR content — interfaces, effects, foliage — tends to have a great deal of it.

Forward+ and clustered forward supply many dynamic lights without a G-buffer, which is why several VR-focused renderers moved back to forward architectures after the industry had largely settled on deferred. **This is a genuine architectural decision with wide consequences**, not a tuning switch, and it is one of the harder things to change later.

DYNAMIC RESOLUTION IS NOT OPTIONAL

Content cost is not uniform across views. Facing a wall is cheap; facing the detailed vista with all the transparency is expensive. **And the user chooses which**, continuously, without warning (Module 01).

A fixed render resolution therefore means either leaving headroom unused in the cheap views or missing frames in the expensive ones — and missing frames is the one thing that is genuinely unacceptable.

So scale the render target resolution per frame to hit the budget, allowing image quality rather than frame time to be the variable. This is the correct trade in VR and the opposite of the usual instinct.

Change it smoothly. A sudden resolution change is visible as a pop in sharpness; ramping over several frames is essentially undetectable. **This is the detail implementations most often get wrong** — the mechanism works and the transitions are distracting.

- **Start from the frame budget** — 11.1 ms at 90 Hz — and subtract the compositor's share, leaving roughly 9 ms for the application.
- **Allocate explicitly:** a millisecond for shadows, so much for opaque geometry, so much for transparency, so much for post-processing. An unallocated budget is an overspent one.
- **Leave about 20% headroom.** The expensive view exists, and the user will find it within the first minute.
- **Measure per eye and in the worst case**, not the average. The average view's cost tells you nothing about whether you will drop frames.
- **Profile on the target hardware.** Standalone headsets are mobile GPUs with mobile thermal envelopes, and **they throttle** — a scene that holds its budget for two minutes may fail after ten, which is a failure mode that does not exist on a desktop GPU and is not visible in a short test.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Unreal Engine — XR performance and rendering documentation (free)	Instanced stereo, multiview, and the engine's VR rendering path. What you will actually configure.
Meta — fixed foveated rendering and performance guidelines (free)	Fixed foveation on mobile hardware, with the measured savings of \$2.
Patney et al. — Towards Foveated Rendering for Gaze-Tracker Virtual Reality (free)	The perceptual basis of foveated rendering and the saccade constraint in \$2.
NVIDIA — Variable Rate Shading documentation (free)	VRS in practice, including the shading-rate image interface.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure the cost of sequential stereo against instanced stereo in your engine. Report CPU and GPU time separately.
- 2 Enable multiview and measure the vertex processing saved. Then make the scene fragment-bound and measure again — the benefit should largely disappear.
- 3 Enable fixed foveated rendering at several strengths. Measure the saving and look for the boundary between shading rates.
- 4 Implement variable rate shading driven by eccentricity and measure the GPU time saved at each strength.
- 5 Drive VRS by motion instead and compare.
- 6 Compare MSAA and TAA while turning your head quickly. Describe the difference during motion rather than when stationary.
- 7 Implement dynamic resolution scaling with a smooth ramp. Then make the ramp instant and confirm the pop is visible.
- 8 Build a written frame budget for a scene, allocate it by stage, and then measure whether the allocation held.
- 9 Run a standalone headset for fifteen minutes and plot frame time. Identify where thermal throttling begins.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name four stereo rendering techniques and say what each saves.
- 2 What can be shared between eyes, what cannot, and what bounds the saving?
- 3 Compare fixed and eye-tracked foveation on assumption, hardware, and saving.
- 4 Why must foveated rendering beat the saccade, and what helps?
- 5 What does variable rate shading coarsen, what stays full rate, and why does that matter?
- 6 Give five ways VR reverses standard rendering practice.
- 7 Give three reasons forward rendering returned for VR.
- 8 Why is dynamic resolution necessary, and what must be done smoothly?
- 9 Give five rules for building a VR frame budget.